

GRAPHS

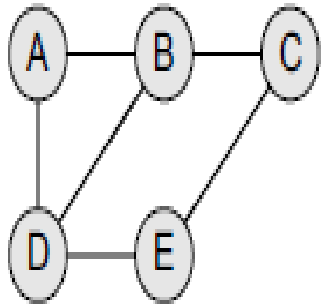
Graph Definition

A graph G is defined as an ordered set (V, E) , where

$V(G)$ represents the set of vertices and $E(G)$ represents the edges that connect these vertices.

$V(G) = \{A, B, C, D \text{ and } E\}$

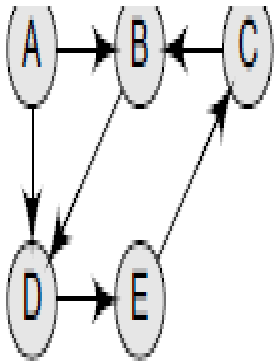
$E(G) = \{(A, B), (B, C), (A, D), (B, D), (D, E), (C, E)\}$.



There are five vertices or nodes and six edges in the graph.

A graph can be directed or undirected.

In an **undirected graph**, edges do not have any direction associated with them. That is, nodes can be traversed from A to B as well as from B to A.



In a **directed graph**, edges form an ordered pair. If there is an edge from A to B, then there is a path from A to B but not from B to A.

The edge (A, B) is said to initiate from node A (also known as initial node) and terminate at node B (terminal node).

Graph Terminology

Adjacent nodes or neighbors

For every edge, $e = (u, v)$ that connects nodes u and v , the nodes u and v are the end-points and are said to be the adjacent nodes or neighbors.

Degree of a node

Degree of a node u , $\deg(u)$, is the total number of edges containing the node u .

Alternatively, Degree is the number of edges connected to a vertex

If **$\deg(u) = 0$** , it means that u does not belong to any edge and such a node is known as an ***isolated node***.

Path A path P written as $P = \{v_0, v_1, v_2, \dots, v_n\}$, of length n from a node u to v is defined as a sequence of $(n+1)$ nodes. Here, $u = v_0$, $v = v_n$ and v_{i-1} is adjacent to v_i for $i = 1, 2, 3, \dots, n$.

Closed path A path P is known as a closed path if the **edge has the same end-points**. That is, if $v_0 = v_n$.

Cycle A path in which the first and the last vertices are same. A *simple cycle* has no repeated edges or vertices (except the first and last vertices).

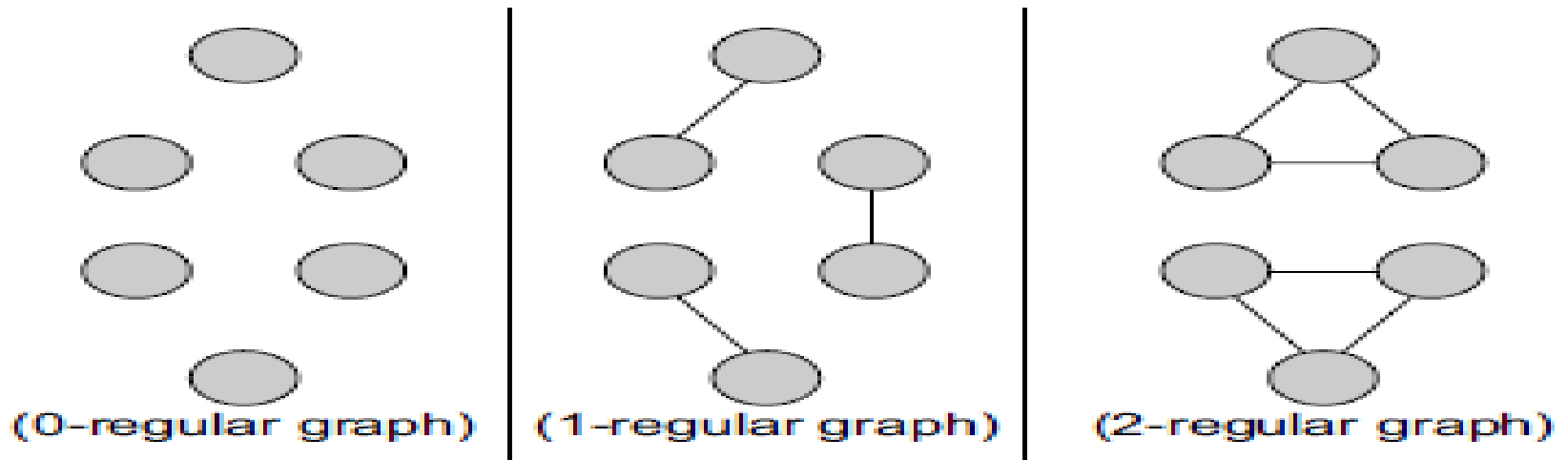
Simple path A path P is known as a simple path if all the nodes in the path are distinct with an exception that v_0 may be equal to v_n . If $v_0 = v_n$, then the path is called a closed simple path.

Regular graph:

It is a graph where each vertex has the same number of neighbors. That is, every node has the same degree.

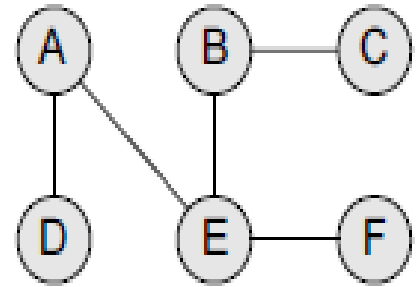
A regular graph with vertices of degree k is called a k -regular graph or a regular graph of degree k .

Figure shows regular graphs.

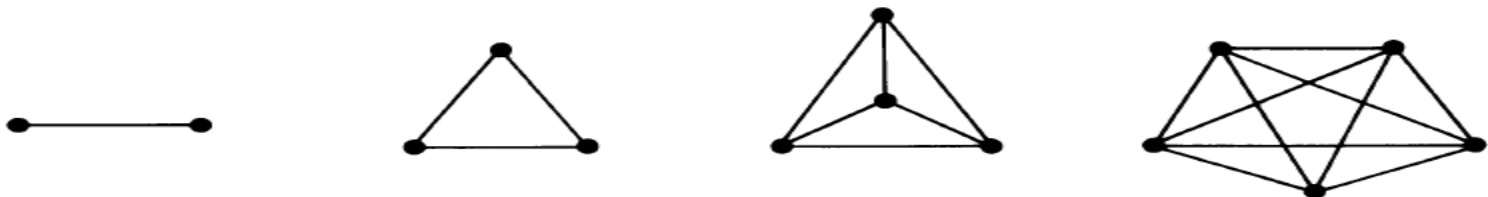


Connected graph A graph is said to be connected if for any two vertices (u, v) in V there is a path from u to v . That is to say that *there are no isolated nodes* in a connected graph.

A connected graph that does not have any cycle is called a tree. Therefore, a tree is treated as a special graph.

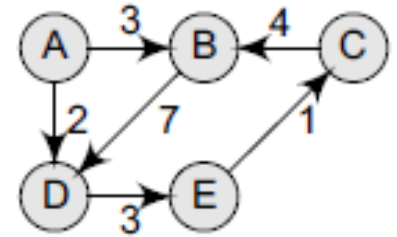


Complete graph A graph G is said to be complete if all its nodes are fully connected. That is, there is a *path from one node to every other node* in the graph. A complete graph *has $n(n-1)/2$ edges*, where n is the number of nodes in G.



Labelled graph or weighted graph.

In a weighted graph, the edges of the graph are assigned some weight or length. The weight of an edge denoted by $w(e)$ is a positive value which indicates the cost of traversing the edge.

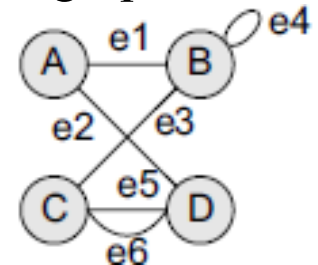


Multiple edges Distinct edges which connect the same end-points are called multiple edges.

That is, $e = (u, v)$ and $e' = (u, v)$ are known as multiple edges of G .

Loop An edge that has identical end-points is called a loop. That is, $e = (u, u)$.

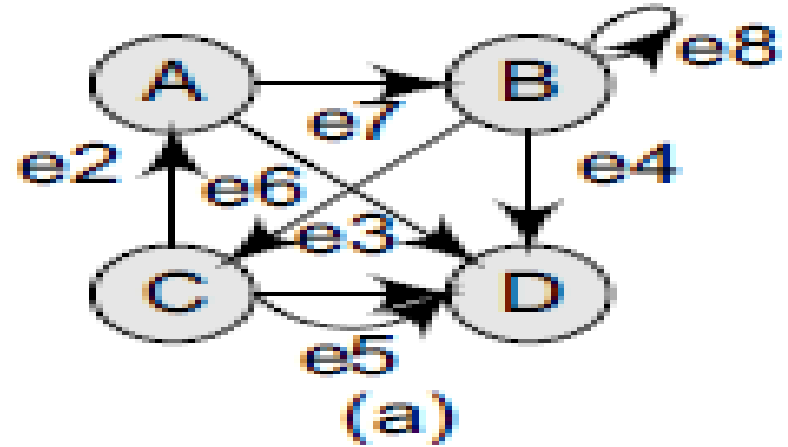
Multi-graph A graph with multiple edges and/or loops is called a multi-graph.



Size of a graph The size of a graph is the total number of edges in it.

Directed Graphs

A directed graph G , also known as a *digraph*, is a graph in which every edge has a direction assigned to it. An edge of a directed graph is given as an ordered pair (u, v) of nodes in G .



For an edge (u, v) ,

1. The edge begins at u and terminates at v .
2. u is known as the origin or initial point of e . Correspondingly, v is known as the destination or terminal point of e .
3. u is the predecessor of v . Correspondingly, v is the successor of u .
4. Nodes u and v are adjacent to each other.

Terminology of a Directed Graph

Out-degree of a node The out-degree of a node u , written as $\text{outdeg}(u)$, is the number of edges that originate from u .

In-degree of a node The in-degree of a node u , written as $\text{indeg}(u)$, is the number of edges that terminate at u .

Degree of a node The degree of a node, written as $\text{deg}(u)$, is equal to the sum of in-degree and out-degree of that node.

Therefore, **$\text{deg}(u) = \text{indeg}(u) + \text{outdeg}(u)$** .

Isolated vertex A vertex with degree zero. Such a vertex is not an endpoint of any edge.

Pendant vertex (also known as leaf vertex) A vertex with degree one.

Transitive Closure of a Directed Graph

For a directed graph $G = (V, E)$, the transitive closure of G is a graph $G^* = (V, E^*)$. In G^* , for every vertex pair v, w in V there is an edge (v, w) in E^* if and only if there is a valid path from v to w in G .

Where and Why is it Needed?

Finding the transitive closure of a directed graph is an important problem in the following computational tasks:

- Transitive closure is used to find the reachability analysis of transition networks representing distributed and parallel systems.
- It is used in the construction of parsing automata in compiler construction.
- transitive closure computation is being used to evaluate recursive database queries (because almost all practical recursive queries are transitive in nature).

Algorithm for finding Transitive Closure

Transitive_Closure(A, t, n)

Step 1: SET i=1, j=1, k=1

Step 2: Repeat Steps 3 and 4 while i<=n

Step 3: Repeat Step 4 while j<=n

Step 4: IF (A[i][j] = 1)
SET t[i][j] = 1

ELSE

SET t[i][j] = 0

INCREMENT j

[END OF LOOP]

INCREMENT i

[END OF LOOP]

Step 5: Repeat Steps 6 to 11 while k<=n

Step 6: Repeat Steps 7 to 10 while i<=n

Step 7: Repeat Steps 8 and 9 while j<=n

Step 8: SET t[i,j] = t[i][j] V (t[i][k] ∧ t[k][j])

Step 9: INCREMENT j

[END OF LOOP]

Step 10: INCREMENT i

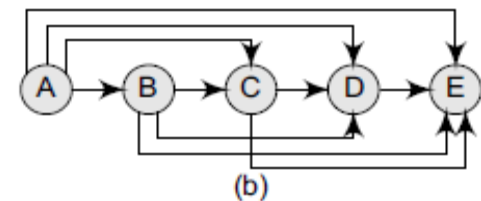
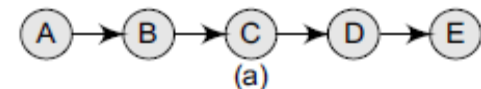
[END OF LOOP]

Step 11: INCREMENT k

[END OF LOOP]

Step 12: END

When $k = 0$	$T_{ij}^0 = \begin{cases} 0 & \text{if } (i, j) \text{ is not in } E \\ 1 & \text{if } (i, j) \text{ is in } E \end{cases}$
When $k \geq 1$	$T_{ij}^k = T_{ij}^{k-1} \vee (T_{ik}^{k-1} \wedge T_{kj}^{k-1})$



(a) A graph G and its
(b) transitive closure
 G^*

REPRESENTATION OF GRAPHS

There are two common ways of storing graphs in the computer's memory. They are:

1. *Sequential representation* by using an adjacency matrix.
2. *Linked representation* by using an adjacency list that stores the neighbors of a node using a linked list.

Adjacency Matrix Representation

An adjacency matrix is used to **represent which nodes are adjacent to one another**.

By definition, two nodes are said to be adjacent if there is an edge connecting them.

$$a_{ij} = \begin{cases} 1 & \text{[if } v_i \text{ is adjacent to } v_j, \text{ that is} \\ & \text{there is an edge } (v_i, v_j)] \\ 0 & \text{[otherwise]} \end{cases}$$

In a directed graph G , if node v is adjacent to node u , then there is definitely an edge from u to v . That is, if v is adjacent to u , we can get from u to v by traversing one edge.

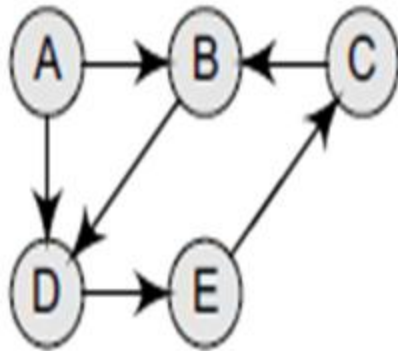
For any graph **G having n nodes**, the adjacency matrix will have the dimension of $n \times n$.

Adjacency Matrix Representation

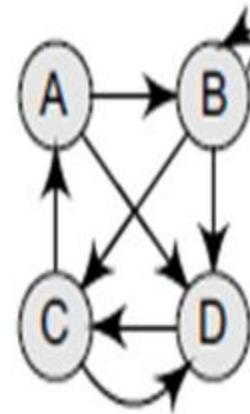
An adjacency matrix is used to **represent which nodes are adjacent to one another**.

- In an adjacency matrix, the rows and columns are labeled by graph vertices.
- An entry a_{ij} in the adjacency matrix will contain 1, if vertices v_i and v_j are adjacent to each other.
- If the nodes are not adjacent, a_{ij} will be set to zero.
- Since an adjacency matrix **contains only 0s and 1s**, it is called a *bit matrix* or a *Boolean matrix*.
- The entries in the matrix depend on the ordering of the nodes in G . Therefore, a change in the order of nodes will result in a different adjacency matrix.

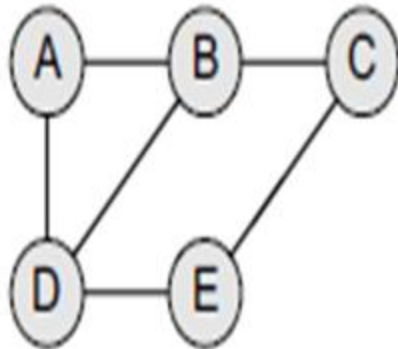
Examples - Adjacency Matrix Representation



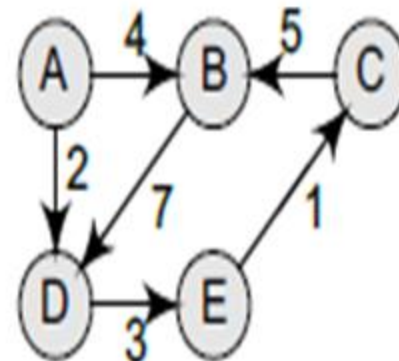
(a) Directed graph



(b) Directed graph with loop

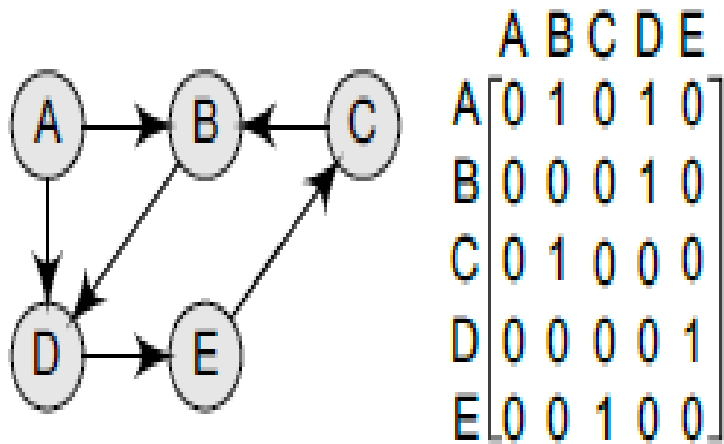


(c) Undirected graph

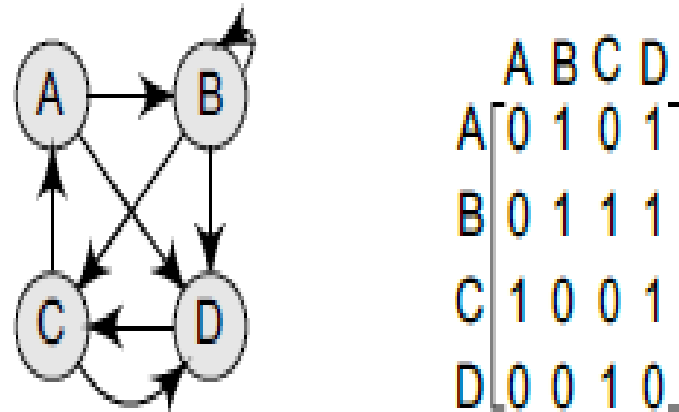


(d) Weighted graph

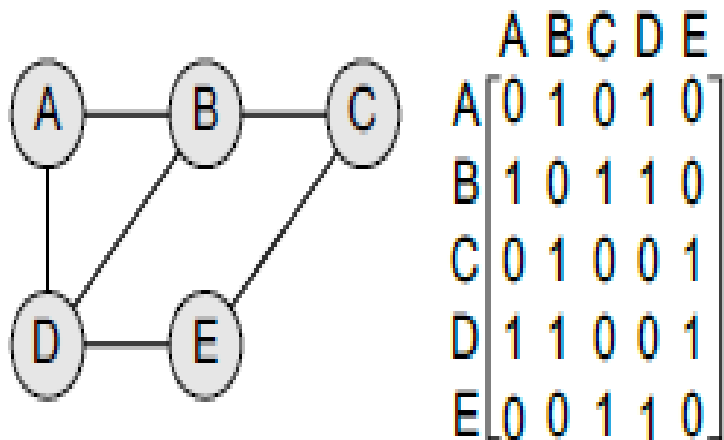
Examples - Adjacency Matrix Representation



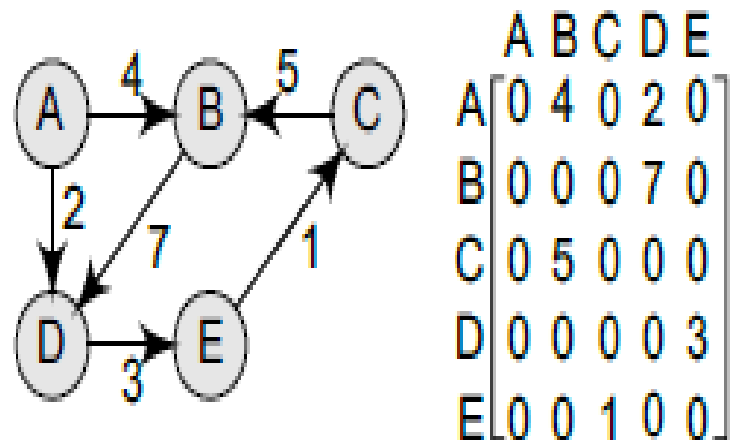
(a) Directed graph



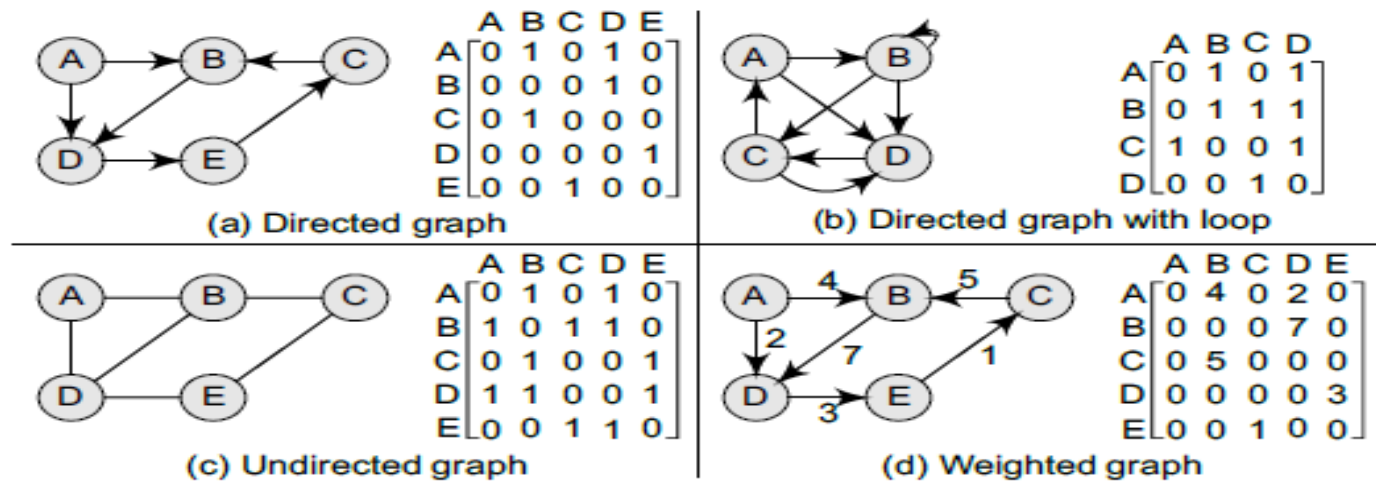
(b) Directed graph with loop



(c) Undirected graph



(d) Weighted graph



From the above examples, we can draw the following conclusions:

- For a simple graph (that has no loops), the adjacency matrix has 0s on the diagonal.
- The adjacency matrix of an undirected graph is symmetric.
- The memory use of an adjacency matrix is $O(n^2)$, where n is the number of nodes in the graph.
- Number of 1s (or non-zero entries) in an adjacency matrix is equal to the number of edges in the graph.
- The adjacency matrix for a weighted graph contains the weights of the edges connecting the nodes.

Adjacency List Representation

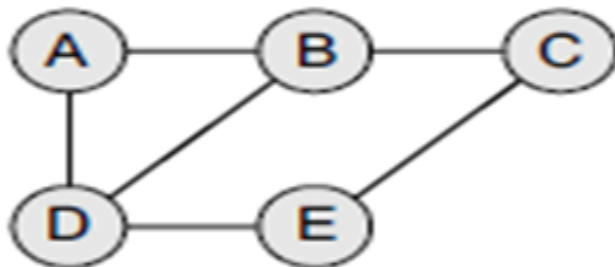
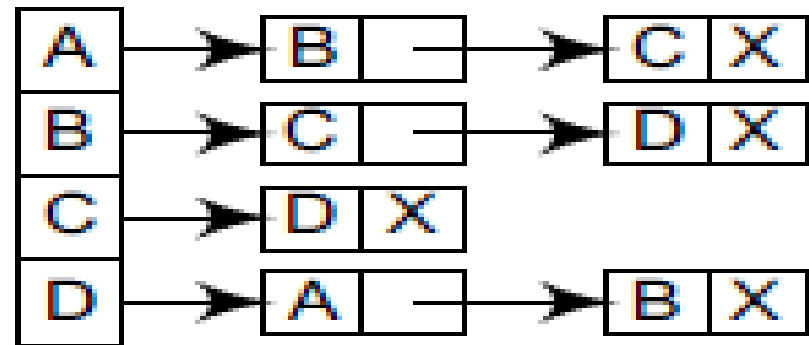
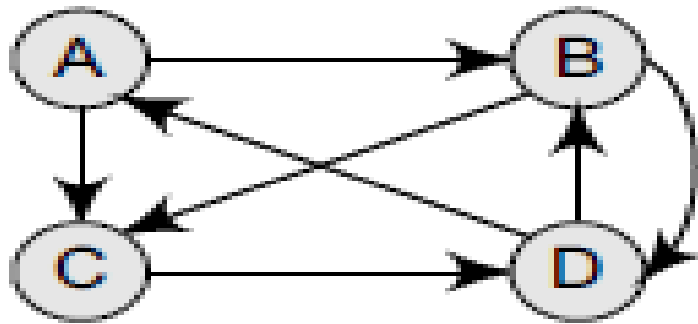
This structure **consists of a list of all nodes in G**.

Every node is in turn linked to its own list that contains the names of all other nodes that are adjacent to it.

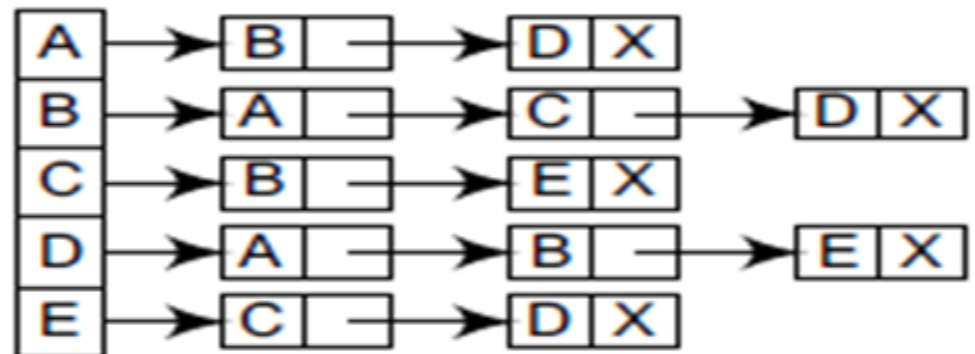
The key advantages of using an adjacency list are:

1. It is easy to follow and clearly shows the adjacent nodes of a particular node.
2. It is often used for storing **graphs that have a small-to-moderate number of edges**. That is, an adjacency list is preferred for representing sparse graphs in the computer's memory; otherwise, an adjacency matrix is a good choice.
3. Adding new nodes in G is easy and straightforward when G is represented using an adjacency list.
4. Adding new nodes in an adjacency matrix is a difficult task, as the size of the matrix needs to be changed and existing nodes may have to be reordered.

Examples- Adjacency List Representation



(Undirected graph)

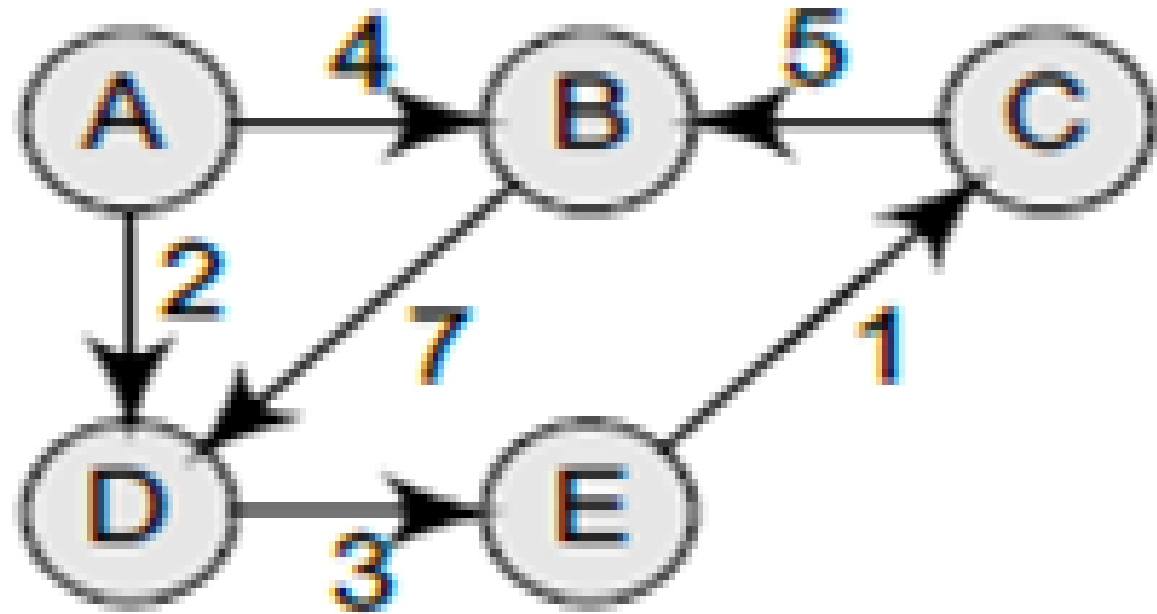


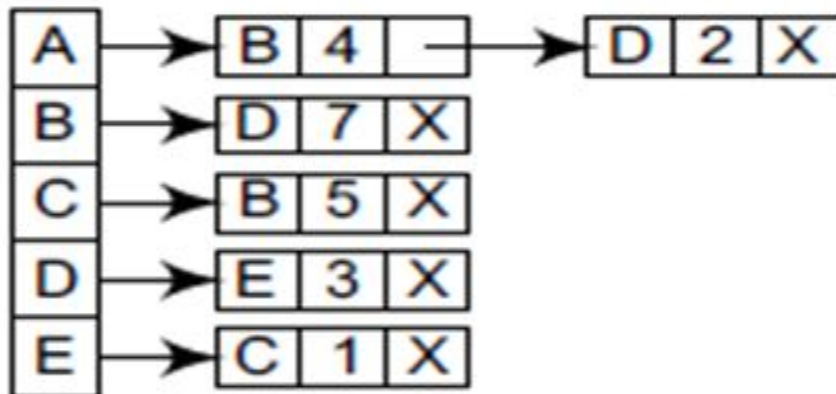
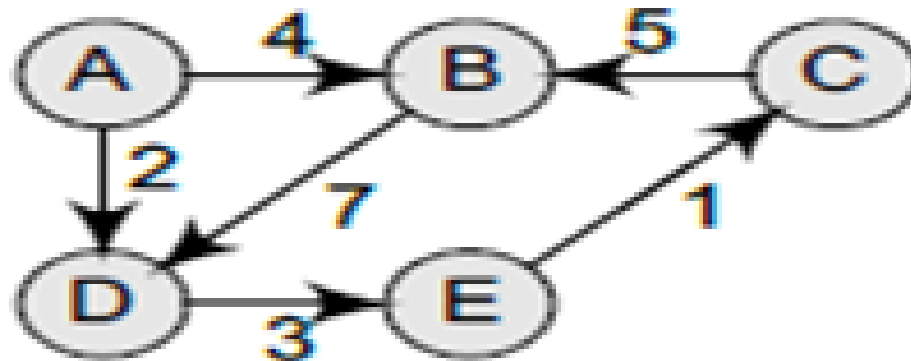
Adjacency list for an undirected graph
and a weighted graph

Represent the graph using

1. Adjacency list

2. Adjacency Matrix

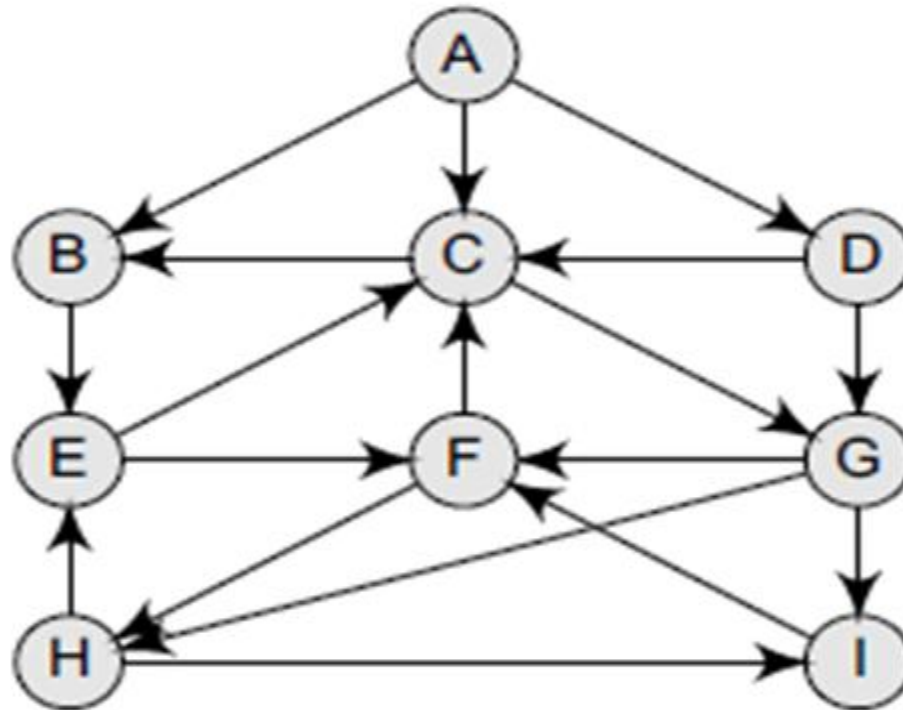


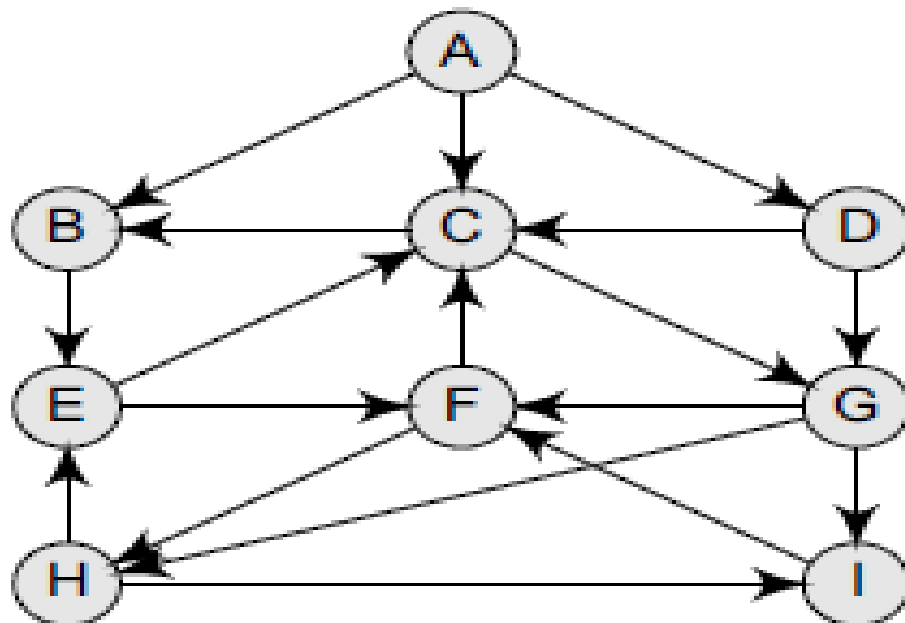


	A	B	C	D	E
A	0	4	0	2	0
B	0	0	0	7	0
C	0	5	0	0	0
D	0	0	0	0	3
E	0	0	1	0	0

Represent the graph using

1. Adjacency list
2. Adjacency Matrix





Adjacency lists

A: B, C, D

B: E

C: B, G

D: C, G

E: C, F

F: C, H

G: F, H, I

H: E, I

I: F